

Санкт-Петербургский государственный университет

Направление "Большие данные и распределенная цифровая платформа"

**Лабораторная работа по дисциплине
Алгоритмы и структуры данных**

**"Решение задачи о коммивояжере с помощью метода
имитации отжига."**

Выполнил:

Зайнуллин Мансур Альбертович

Группа: 23.Б16-пу

Руководитель:

Дик Александр Геннадьевич

ассистент кафедры компьютерного
моделирования

и многопоточных систем

Санкт-Петербург

2025

Оглавление

1	Цель работы	3
2	Теоретические основы	4
2.1	Формализация задачи коммивояжера	4
2.2	Алгоритм имитации отжига	4
2.3	Сверхбыстрый отжиг	5
3	Описание программы	7
3.1	Структура проекта	7
3.2	Спецификация программы	7
3.2.1	Модуль graph.py	8
3.2.2	Модуль simulated_annealing.py	10
3.2.3	Модуль DFS.py	10
3.2.4	Модуль colorGUI.py	10
3.2.5	Модуль main.py	11
4	Контрольный пример	12
5	Рекомендации пользователю	13
5.1	Установка программы	13
5.2	Использование программы	14
5.2.1	Создание графа	14
5.2.2	Настройка параметров	14
5.2.3	Запуск алгоритма	14
5.2.4	Результаты	14
5.2.5	Очистка графа	14
6	Тестирование алгоритма и его модификации	15
6.1	Тестирование параметра	15
6.2	Тестирование на графах разных размеров	17
6.2.1	Граф с 4 вершинами и 8 рёбрами	17
6.2.2	Граф с 20 вершинами и 186 рёбрами	19
6.2.3	Граф с 100 вершинами и 3395 рёбрами	21

6.2.4	Граф с 500 вершинами и 100000 рёбрами	24
6.3	Анализ результатов	27
6.4	Сравнение алгоритма имитации отжига с методом ближайшего соседа	28
7	Вывод	30
8	Полезные ссылки	31
9	Листинг программы	32
9.1	main.py	33
9.2	DFS.py	35
9.3	simulated _a nnealing.py	38
9.4	colorGUI.py	43

1 Цель работы

Целью данной работы является исследование особенностей решения задачи о коммивояжере с помощью алгоритма имитации отжига.

2 Теоретические основы

2.1 Формализация задачи коммивояжера

Задача коммивояжера (TSP) в ориентированном графе $G = (V, A)$ с весами дуг $w(a)$, $a \in A$, заключается в нахождении гамильтонова цикла C минимальной длины:

$$L(C) = \sum_{a \in C} w(a) \rightarrow \min,$$

где V — множество вершин, A — множество дуг. Цикл должен проходить через каждую вершину ровно один раз и возвращаться в начальную точку.

2.2 Алгоритм имитации отжига

Алгоритм имитации отжига (Simulated Annealing, SA) решает задачу оптимизации через моделирование физического процесса отжига. Для TSP решение S — гамильтонов цикл, целевая функция — длина пути $L(S)$.

- **Инициализация:** Начальный путь S_{current} строится с помощью DFS. Начальная длина $L_{\text{current}} = L(S_{\text{current}})$, температура $T = T_0$ (по умолчанию $T_0 = 100$).
- **Генерация соседа:** Применяется 2-орт перестановка: выбираются два индекса $a < b$, участок пути $[a, b]$ инвертируется, формируя S_{new} с длиной L_{new} .
- **Критерий принятия:** Разность длин $\Delta L = L_{\text{new}} - L_{\text{current}}$. Новое решение принимается, если:

$$\Delta L < 0 \quad \text{или} \quad \text{random}() < \exp \left(-\frac{\Delta L}{T \cdot \left(1 + 0.1 \cdot \frac{L_{\text{current}}}{L_{\text{best}}} \right)} \right),$$

где L_{best} — длина лучшего найденного пути. Этот критерий адаптирует вероятность в зависимости от прогресса.

- **Охлаждение:** Температура обновляется по графику:

$$T \leftarrow T \cdot \alpha \quad (\alpha = 0.95) \quad \text{или} \quad T = \frac{T_0}{1 + k} \quad (\text{при сверхбыстром отжиге}),$$

где k — номер итерации.

- **Остановка:** Цикл завершается при $T < T_{\min}$ ($T_{\min} = 1$).

Псевдокод:

```

S_current <- DFS(G)
L_current <- L(S_current), S_best <- S_current, L_best <- L_current
T <- T_0, k <- 0
while T >= T_min:
    k <- k + 1
    S_new <- two_opt_swap(S_current, a, b) #      a < b
    L_new <- L(S_new)
    if L_new < infinity:
        Delta L <- L_new - L_current
        if Delta L < 0 or random() < exp(-Delta L / (T * (1 + 0.1 *
            ↪ L_current / L_best)))):
            S_current <- S_new, L_current <- L_new
            if L_current < L_best:
                S_best <- S_current, L_best <- L_current
    if fast_annealing:
        T <- T_0 / (1 + k)
    else:
        T <- T * alpha
return S_best, L_best

```

2.3 Сверхбыстрый отжиг

В режиме сверхбыстрого отжига температура снижается по формуле:

$$T = \frac{T_0}{1 + k},$$

в отличие от стандартного геометрического охлаждения $T \leftarrow T \cdot \alpha$.

Это ускоряет сходимость за счёт быстрого уменьшения T , но может снизить качество решения из-за ограниченного исследования пространства. Флаг `fast_annealing` переключает режимы.

3 Описание программы

Программа разработана для решения задачи коммивояжера с использованием алгоритма имитации отжига и включает графический интерфейс для визуализации графа и результатов. Реализация выполнена на языке Python 3 с применением библиотеки Tkinter для интерфейса пользователя.

3.1 Структура проекта

Проект состоит из следующих модулей, каждый из которых выполняет строго определенную функцию:

- `main.py`: Точка входа программы. Отвечает за инициализацию графа и запуск графического интерфейса.
- `colorGUI.py`: Реализует графический интерфейс с использованием Tkinter. Обеспечивает визуализацию и взаимодействие пользователя.
- `graph.py`: Содержит объектно-ориентированную реализацию графа через классы `Graph`, `Node` и `Edge`.
- `simulated_annealing.py`: Реализует алгоритм имитации отжига с модификацией сверхбыстрого отжига.
- `DFS.py`: Предоставляет функцию поиска начального гамильтонова цикла с использованием DFS.

Модули взаимодействуют следующим образом: `main.py` создает граф и передает его в `colorGUI.py`, который вызывает `simulated_annealing.py` для выполнения алгоритма, используя `DFS.py` для инициализации пути.

3.2 Спецификация программы

Спецификация описывает ключевые компоненты программы, включая их назначение, входные и выходные данные. Описание основано на документации кода.

3.2.1 Модуль `graph.py`

Класс `Node`

Table 1: Класс `Node`

Метод/Атрибут	Описание	Входные/Выходные данные
<code>value</code>	Уникальный идентификатор вершины	<code>—</code> $\rightarrow$ <code>int</code>
<code>neighbors</code>	Список смежных вершин	<code>—</code> $\rightarrow$ <code>List[Node]</code>
<code>outgoing_edges</code>	Исходящие рёбра	<code>—</code> $\rightarrow$ <code>List[Edge]</code>
<code>x, y</code>	Координаты для визуализации	<code>—</code> $\rightarrow$ <code>float</code> или <code>None</code>
<code>visited</code>	Флаг посещения вершины	<code>—</code> $\rightarrow$ <code>bool</code>
<code>add_neighbor(node)</code>	Добавляет вершину в список соседей	<code>node (Node)</code> $\rightarrow$ Обновление списка <code>neighbors</code>
<code>sorted_edges()</code>	Возвращает отсортированный список исходящих рёбер по весу	<code>—</code> $\rightarrow$ <code>List[Edge]</code> (отсортированный)

Класс Edge

Table 2: Класс Edge

Метод/Атрибут	Описание	Входные/Выходные данные
from_node	Начальная вершина ребра	— → Node
to_node	Конечная вершина ребра	— → Node
weight	Вес ребра	— → int

Класс Graph

Table 3: Класс Graph

Метод/Атрибут	Описание	Входные/Выходные данные
directed	Флаг ориентированности графа	— → bool (True по умолчанию)
nodes	Список всех вершин графа	— → List[Node]
edges	Список всех рёбер графа	— → List[Edge]
add_node(value)	Создаёт и добавляет новую вершину	value (int) → Созданный объект Node
add_edge(from_node, to_node, weight)	Добавляет направленное ребро между вершинами	from_node (Node), to_node (Node), weight (int) → Создание Edge + обратного ребра для неориентированных графов
display()	Выводит текстовое представление графа	— → Вывод в консоль списков nodes/edges

3.2.2 Модуль simulated_annealing.py

Table 4: Модуль simulated_annealing.py

Объект/Функция	Описание	Входные/Выходные данные
calculate_path_length	Вычисляет длину гамильтонова цикла	graph (Graph), path (List[Node]) → int
get_initial_path	Генерирует начальный путь через DFS	graph (Graph) → List[Node]
two_opt_swap	Выполняет 2-opt перестановку для генерации соседнего решения	path (List[Node]), i (int), k (int) → List[Node]
simulated_annealing	Реализует алгоритм имитации отжига с опцией сверхбыстрого отжига	graph (Graph), initial_temp (float, опц.), min_temp (float), cooling_rate (float), fast_annealing (bool) → Tuple[Graph, int, str, str]

3.2.3 Модуль DFS.py

Table 5: Модуль DFS.py

Объект/Функция	Описание	Входные/Выходные данные
dfs_hamiltonian_cycle	Находит любой гамильтонов цикл в графе с использованием DFS	graph (Graph), start_node (Node, опц.) → Tuple[Graph, int, str, str]

3.2.4 Модуль colorGUI.py

Класс LabApp

Table 6: Класс LabApp

Метод/Атрибут	Описание	Входные/Выходные данные
create_widgets()	Инициализирует все элементы интерфейса	— → Построение GUI
on_canvas_click(event)	Обработка кликов: создание вершин/рёбер	event (координаты x,y) → Добавление Node/Edge в граф
start_algorithm()	Запуск алгоритма имитации отжига	— → Обновление выходных данных и визуализации
update_edges_table()	Обновление таблицы рёбер с отметками использованных в решении	— → Синхронизация данных в Treeview
draw_graph()	Отрисовка графа на указанном холсте	canvas, graph → Визуализация вершин и рёбер

3.2.5 Модуль main.py

Table 7: Модуль main.py

Объект/Функция	Описание	Входные/Выходные данные
create_graph()	Генерирует ориентированный граф со случайной топологией	num_nodes (int), num_edges (int), canvas_width (int), canvas_height (int) → Объект Graph
main()	Инициализирует GUI	— → Запуск графического интерфейса

4 Контрольный пример

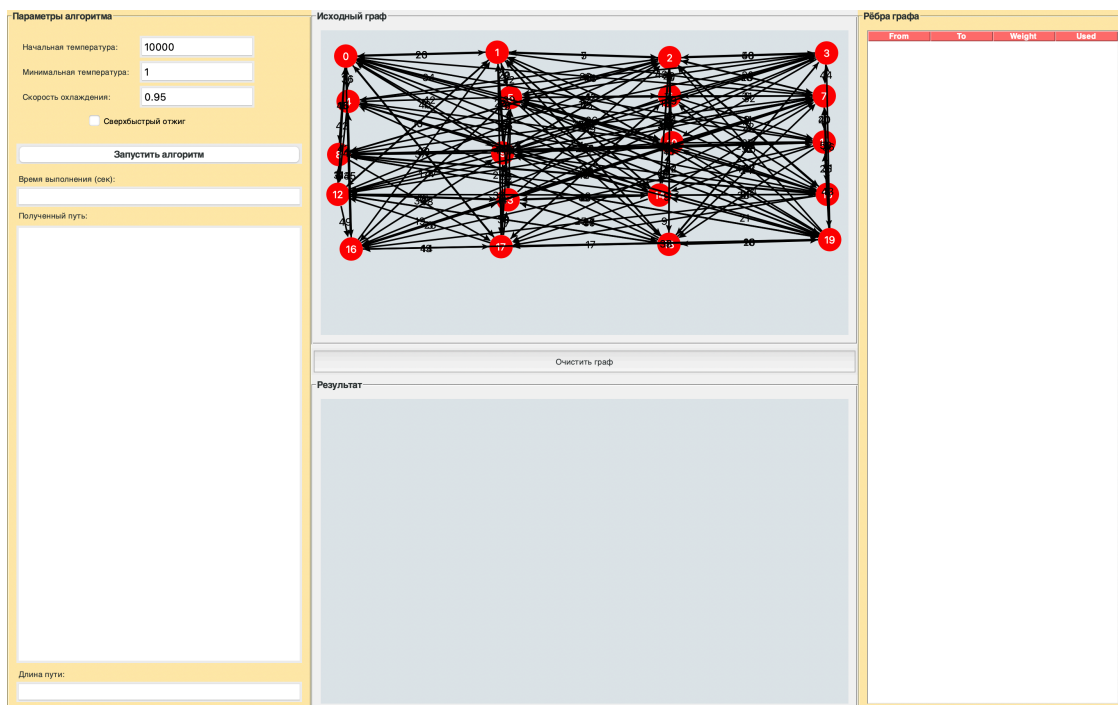


Рис 1. Запуск графического интерфейса через запуск main.py

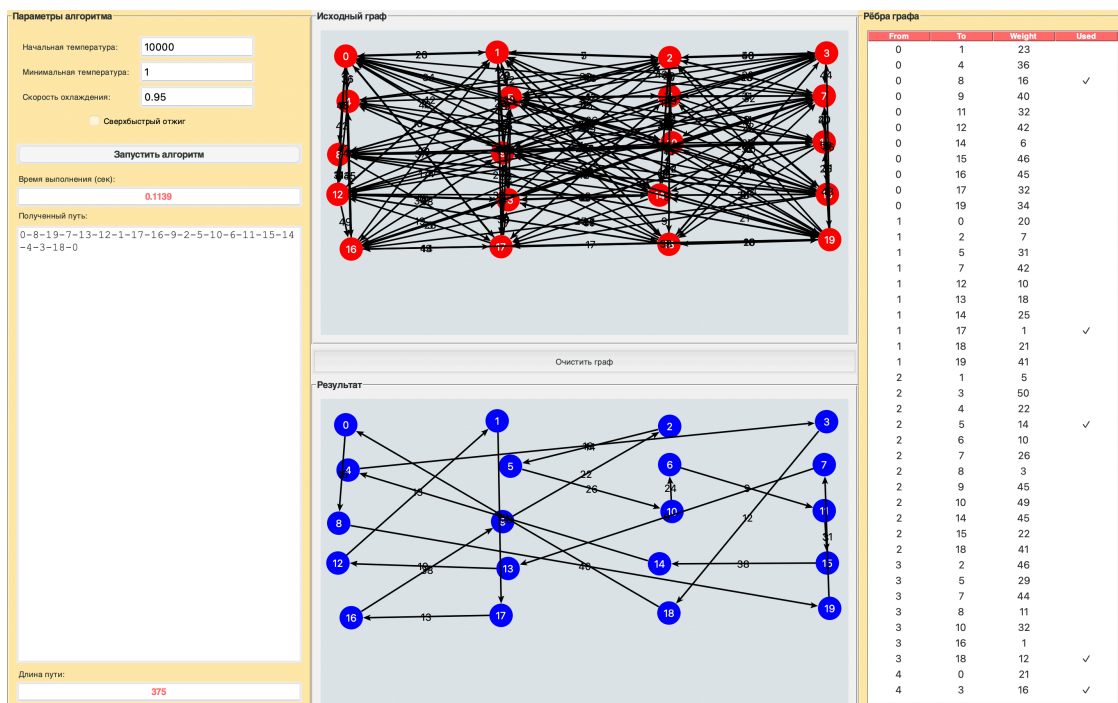


Рис 2. Нажимаем на кнопку ”Запустить алгоритм и почти сразу получаем результат”

5 Рекомендации пользователю

Эта инструкция поможет вам установить и использовать программу. Она написана для Ubuntu, но её легко адаптировать и под другие операционные системы.

5.1 Установка программы

1. **Запустите терминал:** Нажмите `Ctrl + Alt + T` на клавиатуре — откроется окно для ввода команд.
2. **Обновите систему:** Напишите `sudo apt update` и нажмите `Enter`. Введите пароль, если попросят.
3. **Установите Python 3.11:** Введите `sudo apt install python3.11` и нажмите `Enter`.
4. **Установите Git:** Введите `sudo apt install git` и нажмите `Enter`.
5. **Перейдите в домашнюю папку:** Введите `cd ~` и нажмите `Enter`.
6. **Скачайте программу:** Напишите `git clone https://github.com/MansurYa/4-sem-labs-for-algorithms-and-data-s` и нажмите `Enter`.
7. **Войдите в папку программы:** Введите `cd 4-sem-labs-for-algorithms-and-data-structures/Lab2` и нажмите `Enter`.
8. **Установите Tkinter:** Напишите `sudo apt install python3-tk` и нажмите `Enter`.
9. **Запустите программу:** Введите `python3.11 main.py` и нажмите `Enter`.

5.2 Использование программы

После запуска вы увидите окно с настройками слева, графиком в центре и таблицей справа.

5.2.1 Создание графа

- Щёлкните в "Исходный граф" для добавления точки (вершины).
- Щёлкните на две точки подряд, введите вес связи в окне и нажмите ОК.

5.2.2 Настройка параметров

- Настройте гиперпараметры алгоритма в полях, которые находятся верхнем левом углу GUI.

5.2.3 Запуск алгоритма

- Нажмите "Запустить алгоритм" для поиска пути.

5.2.4 Результаты

- В "Полученный путь" увидите порядок точек (например, "0-1-2-0").
- В "Длина пути" — сумма весов связей.
- В "Результат" — рисунок пути. В таблице связи с галочками — это путь.

5.2.5 Очистка графа

- Нажмите "Очистить граф" для нового старта.

6 Тестирование алгоритма и его модификации

В данной главе представлены результаты тестирования базового алгоритма имитации отжига и его модификации (сверхбыстрого отжига) на графах различного размера.

6.1 Тестирование параметра

Методология: Тестирование проводилось на одном и том же ориентированном графе с 250 вершинами и 25000 рёбер. Для алгоритма без модификации и с ней проводился запуск с различными значениями параметра "Скорость охлаждения". Начальная температура $T_0 = 10000$, минимальная температура $T_{\min} = 1$.

Базовый алгоритм

Table 8: Базовый алгоритм

Скорость охлаждения	Длина пути	Время выполнения (с)
0.95	5831	75.7753
0.85	5839	26.3432
0.75	5981	15.3432
0.65	6057	8.8453
0.55	6155	6.0943
0.45	6228	5.5117
0.35	6228	4.3432
0.25	6228	1.7426
0.15	6228	1.6666
0.05	6228	1.6434

Модификация (Сверхбыстрый отжиг)

Table 9: Модификация (Сверхбыстрый отжиг)

Скорость охлаждения	Длина пути	Время выполнения (с)
0.95	5865	4.7755
0.85	5918	4.6034
0.75	6089	4.7114
0.65	6089	4.7315
0.55	6228	5.3431
0.45	6058	5.0032
0.35	6207	5.7154
0.25	6206	4.2994
0.15	6156	4.6774
0.05	6206	4.4009

Вывод: с приближением скорости охлаждения к 1, уменьшается длина найденного пути, но также для базового алгоритма время выполнения возрастает гиперболически.

6.2 Тестирование на графах разных размеров

Методология: Тестирование проводилось на ориентированных взвешенных графах с числом вершин 4, 20, 100 и 500. Для каждого графа выполнено по 5 запусков обоих алгоритмов. Начальная температура $T_0 = 10000$, минимальная температура $T_{\min} = 1$. Базовый алгоритм использует геометрическое охлаждение с $\alpha = 0.95$.

6.2.1 Граф с 4 вершинами и 8 рёбрами

Базовый алгоритм

Table 10: Базовый алгоритм (4 вершины)

№ испытания	Длина пути	Время выполнения (с)
1	112	0.0076
2	112	0.0090
3	112	0.0100
4	112	0.0092
5	112	0.0101

Найденный путь при первом испытании: 0-1-3-2-0

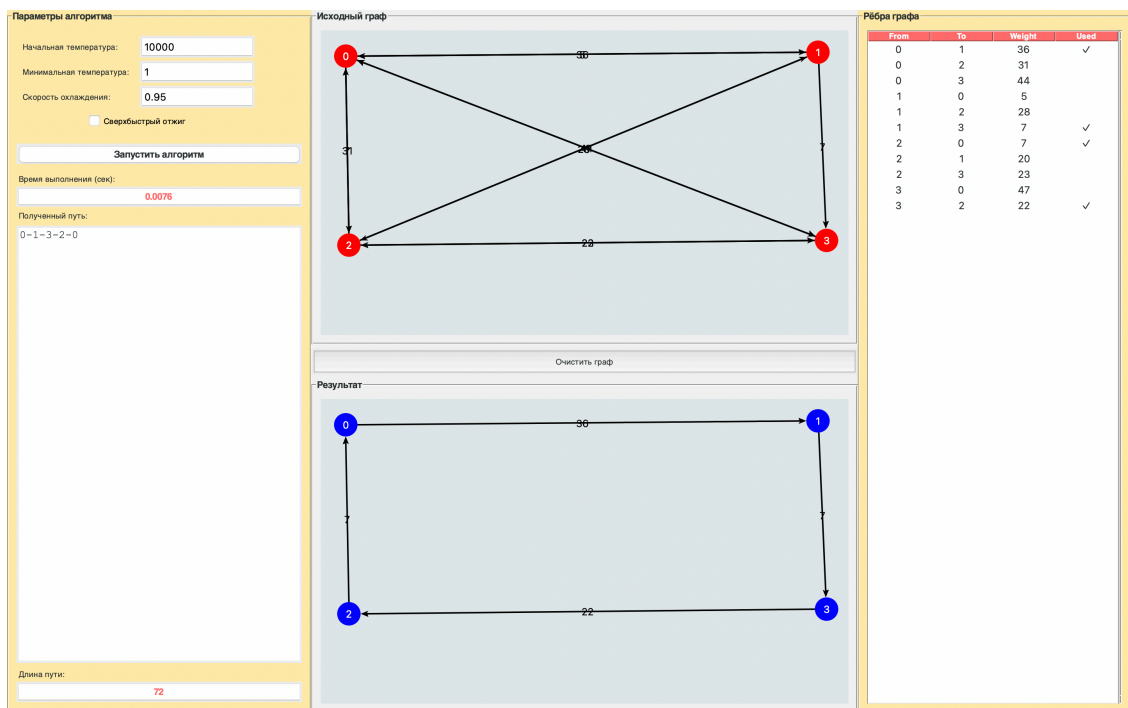


Рис 3. Граф с 4 вершинами (базовый)

Модификация (сверхбыстрый отжиг)

Table 11: Модификация (сверхбыстрый отжиг, 4 вершины)

№ испытания	Длина пути	Время выполнения (с)
1	112	0.1034
2	112	0.1035
3	112	0.1033
4	112	0.1035
5	112	0.1036

Найденный путь при первом испытании: 0-1-3-2-0

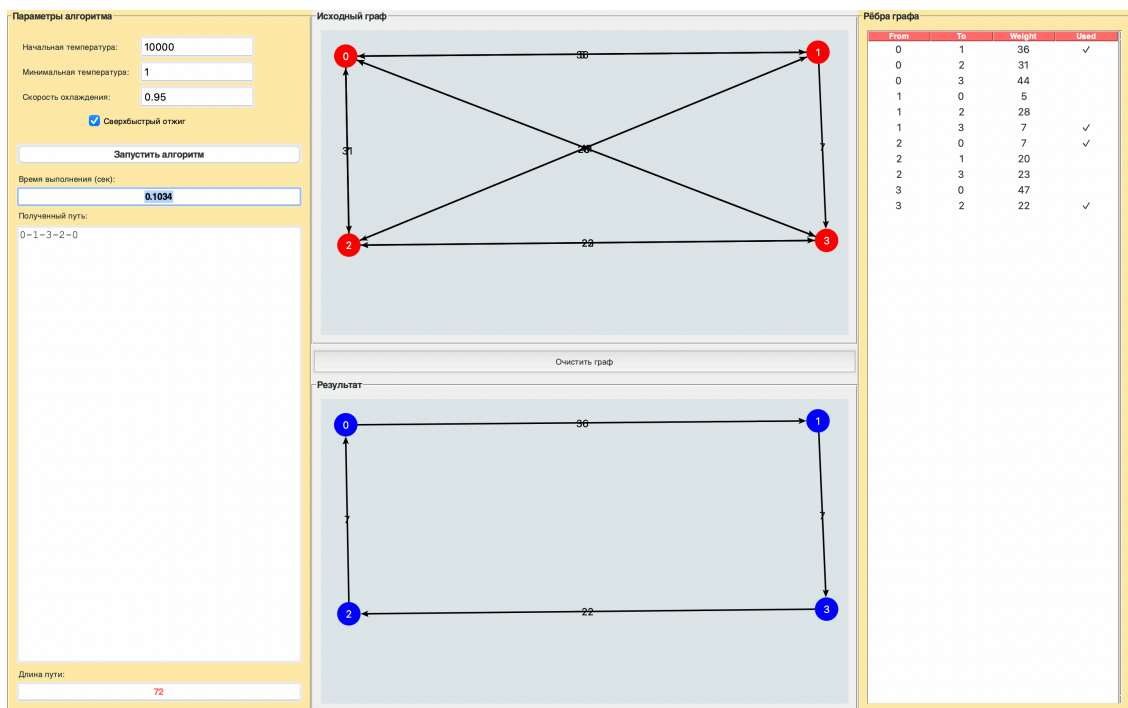


Рис 4. Граф с 4 вершинами (модификация)

6.2.2 Граф с 20 вершинами и 186 рёбрами

Базовый алгоритм

Table 12: Базовый алгоритм (20 вершин)

№ испытания	Длина пути	Время выполнения (с)
1	375	0.1085
2	421	0.1103
3	375	0.1003
4	375	0.1140
5	365	0.1131

Найденный путь при первом испытании: 0-8-19-7-13-12-1-17-16-9-2-5-10-6-11-15-14-4-3-18-0

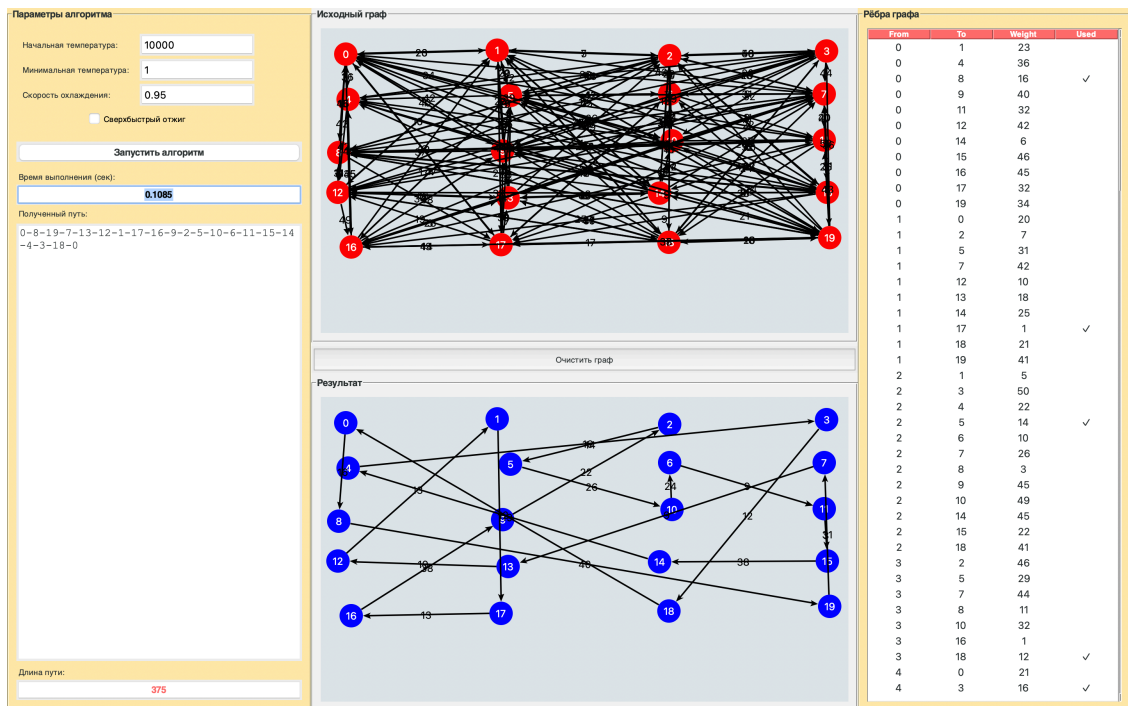


Рис 5. Граф с 20 вершинами (базовый)

Модификация (сверхбыстрый отжиг)

Table 13: Модификация (сверхбыстрый отжиг, 20 вершин)

№ испытания	Длина пути	Время выполнения (с)
1	421	0.1736
2	366	0.1746
3	421	0.1723
4	438	0.1585
5	438	0.1704

Найденный путь при первом испытании: 0-8-19-7-17-16-9-5-13-12-1-2-6-10-11-15-14-4-3-18-0

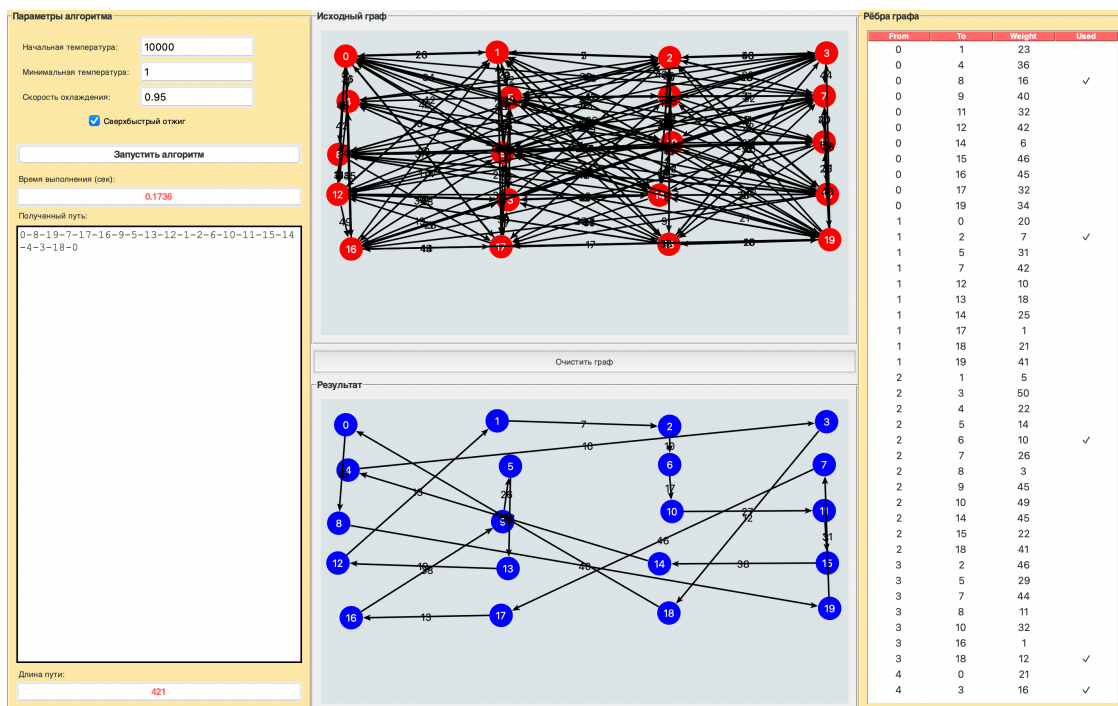


Рис 6. Граф с 20 вершинами (модификация)

6.2.3 Граф с 100 вершинами и 3395 рёбрами

Базовый алгоритм

Table 14: Базовый алгоритм (100 вершин)

№ испытания	Длина пути	Время выполнения (с)
1	2452	11.0466
2	2449	8.5324
3	2449	8.7343
4	2452	10.3518
5	2449	8.7561

Найденный путь при первом испытании: 0-25-39-74-16-19-68-57-43-17-98-82-36-92-76-24-99-70-58-28-91-54-71-33-38-65-64-31-11-73-69-21-90-8-67-83-56-87-30-15-51-34-4-41-53-94-18-37-52-32-85-89-2-6-62-44-66-49-55-59-96-10-12-47-84-93-61-48-72-3-35-95-80-63-7-78-5-22-40-14-1-29-27-42-45-46-60-77-26-9-79-50-81-97-13-88-75-23-86-20-0

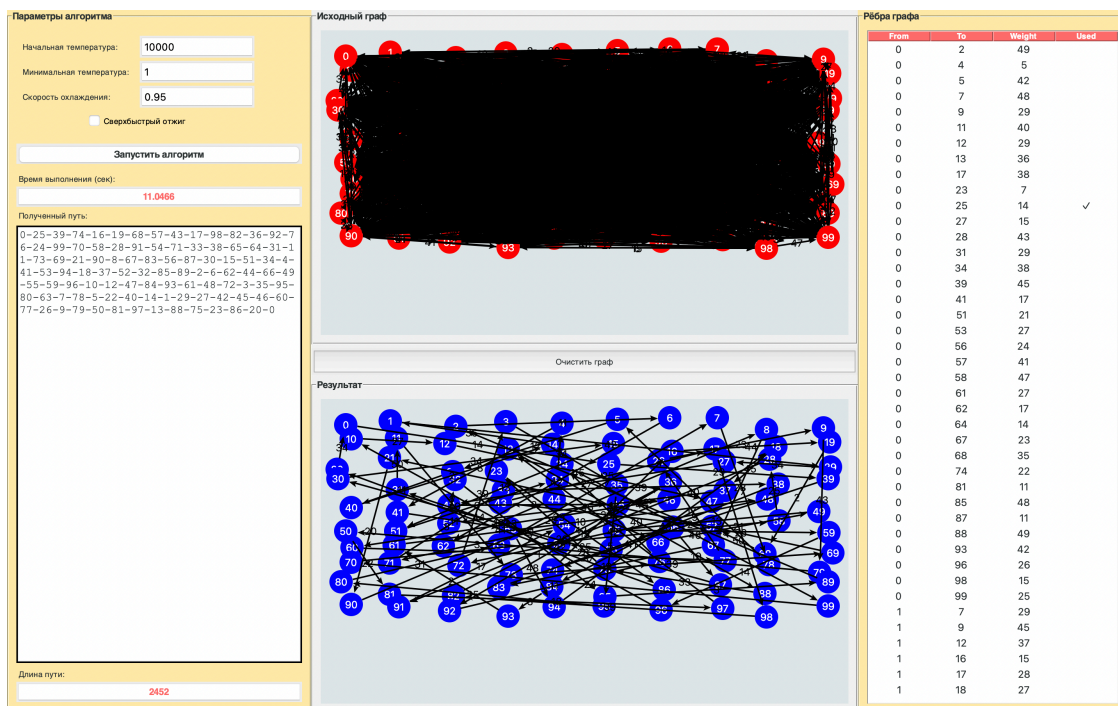


Рис 7. Граф с 100 вершинами (базовый)

Модификация (сверхбыстрый отжиг)

Table 15: Модификация (сверхбыстрый отжиг, 100 вершин)

№ испытания	Длина пути	Время выполнения (с)
1	2489	1.3733
2	2516	1.3534
3	2542	1.4214
4	2472	1.3719
5	2489	1.4125

Найденный путь при первом испытании: 0-25-39-74-16-19-68-57-43-17-98-82-36-92-76-24-99-70-58-28-91-54-71-33-38-65-64-31-11-73-69-21-90-8-67-83-56-87-30-15-51-34-4-41-53-94-18-37-52-32-85-89-2-6-62-44-66-49-55-96-59-10-12-47-84-93-61-48-72-3-35-95-80-63-7-78-5-22-40-14-1-29-27-42-45-46-60-77-26-9-79-50-81-97-13-88-75-23-86-20-0

6.2.4 Граф с 500 вершинами и 100000 рёбрами

Базовый алгоритм

Table 16: Базовый алгоритм (500 вершин)

№ испытания	Длина пути	Время выполнения (с)
1	11739	547.1692
2	11742	534.5432
3	11743	514.4324
4	11734	546.5422
5	11742	538.5432

Найденный путь при первом испытании: 0-266-288-172-125-19-205-40-263-244-223-247-23-243-310-28-186-460-318-145-70-428-...-431-435-204-416-261-155-50-153-322-135-193-449-258-75-484-479-98-150-348-14-158-235-286-37-302-471-112-226-385-182-441-262-132-376-55-446-483-290-221-117-402-245-498-103-346-126-11-469-368-79-214-394-384-137-136-447-268-81-224-29-324-0



Рис 9. Граф с 500 вершинами (базовый)

Модификация (сверхбыстрый отжиг)

Table 17: Модификация (сверхбыстрый отжиг, 500 вершин)

№ испытания	Длина пути	Время выполнения (с)
1	12533	24.3529
2	12483	23.2544
3	12533	23.2134
4	12453	16.4312
5	12321	29.4324

Найденный путь при первом испытании: 0-266-288-172-125-205-19-40-263-244-223-247-243-23-310-28-186-460-318-145-70-428-383-490-453-173-194-337-105-345-...-77-3-431-435-204-416-50-155-261-153-322-135-193-449-258-75-484-479-98-150-348-14-158-235-286-37-302-471-112-226-385-182-441-262-132-376-55-446-221-117-402-245-483-290-498-103-346-126-11-469-368-79-214-394-384-137-136-447-268-81-224-29-324-0

6.3 Анализ результатов

Базовый алгоритм, использующий геометрическое охлаждение ($T \leftarrow T \cdot 0.95$), превосходит модификацию по качеству путей (например, 2450.2 против 2501.6 для 100 вершин), так как медленное снижение температуры расширяет поиск, снижая риск локальных минимумов. Сверхбыстрый отжиг ($T = T_0/(1 + k)$) сокращает время выполнения (23.3 с против 536.2 с для 500 вершин), но быстрый спад температуры ограничивает перебор, увеличивая длину пути на 6–10%. На малых графах (4 вершины) оба алгоритма находят оптимум (112), но модификация менее эффективна (0.1035 с против 0.0092 с) из-за накладных расходов на реализацию. С ростом графа разрыв в скорости растет (до 23 раз для 500 вершин), а вариативность путей в модификации увеличивается (например, 2472–2542 для 100 вершин против 2449–2452 у базового). Тесты на графе с 250 вершин (см. графики ниже) показывают, что при высокой скорости охлаждения ($\alpha = 0.95$) базовый алгоритм точнее (5831 против 5865), но при низкой ($\alpha \leq 0.35$) результаты сближаются (6228 против 6206), а модификация стабильнее по времени (~4.5 с против 1.6–4.3 с).

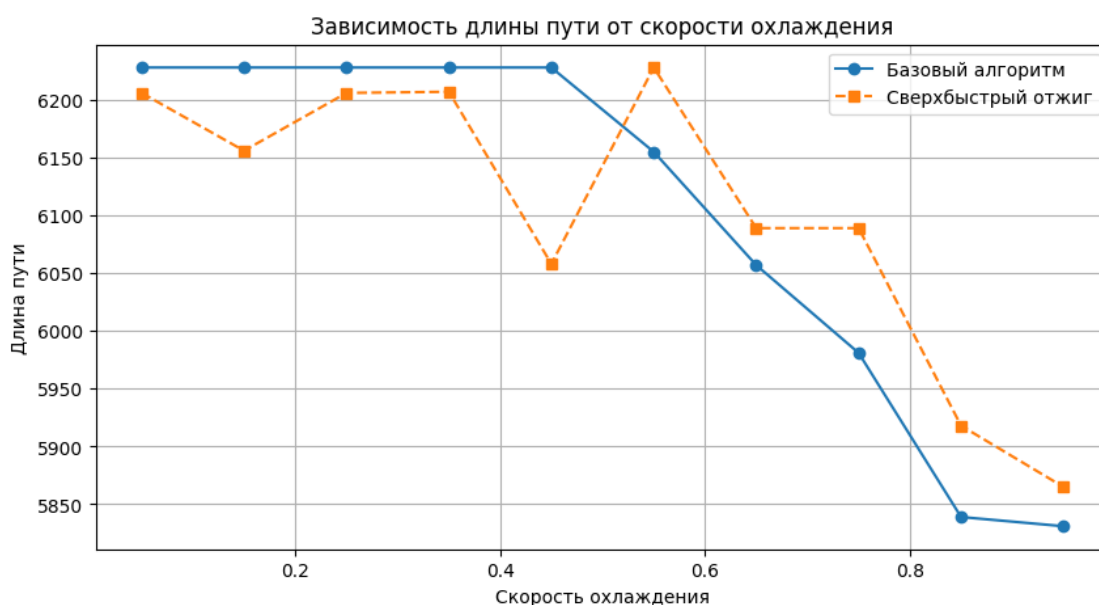


Рис 11. Зависимость длины найденного пути от скорости охлаждения

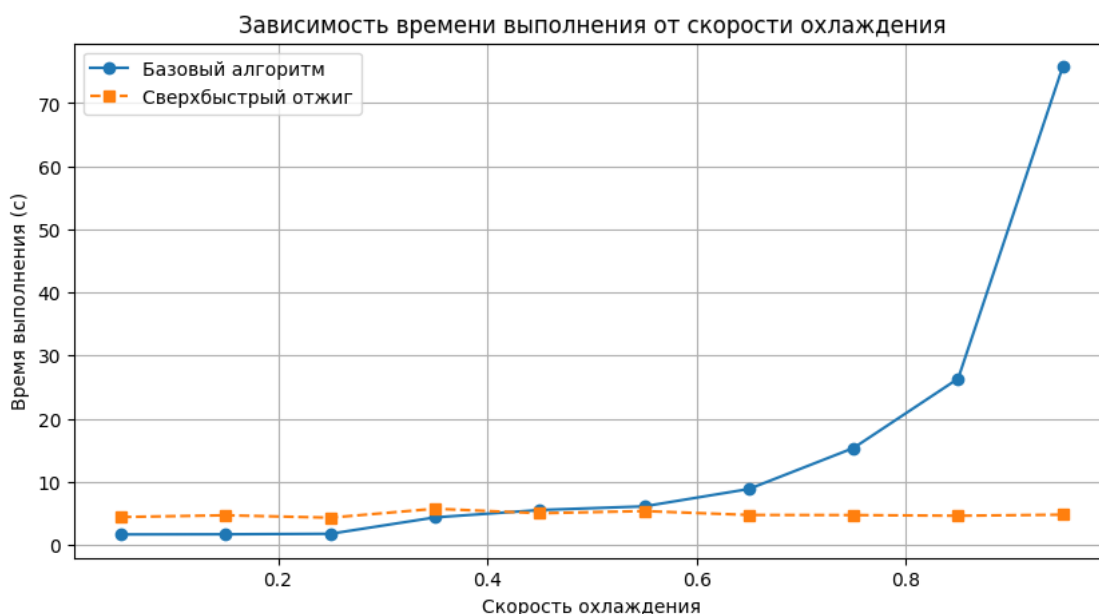


Рис 12. Зависимость времени выполнения от скорости охлаждения

6.4 Сравнение алгоритма имитации отжига с методом ближайшего соседа

Длина пути:

Table 18: Сравнение длины пути

Вершины	Ближ. сосед (баз.)	Ближ. сосед (мод.)	Отжиг (баз.)	Отжиг (сверх.)
4	Нет пути	112	112	112
20	Нет пути	155	382.2	416.8
100	Нет пути	524	2450.2	2501.6
500	Нет пути	863	11740	12464.6

Время выполнения (с):

Модификация метода ближайшего соседа превосходит имитацию отжига по качеству и скорости. Она находит пути в 2.5–14 раз короче на графах от 20 до 500 вершин и работает в десятки–сотни раз быстрее благодаря детерминированному перебору ($O(n^3)$). Базовый отжиг точен только на малых графах, но сильно проигрывает по времени на больших. Сверхбыстрый отжиг ускоряет базовый, но жертвует длиной пути.

Table 19: Сравнение времени выполнения

Вершины	Ближ. сосед (баз.)	Ближ. сосед (мод.)	Отжиг (баз.)	Отжиг (сверх.)
4	0.00016	0.00024	0.00918	0.10346
20	0.00014	0.00188	0.10924	0.16988
100	0.0006	0.03498	9.48424	1.3865
500	0.01534	6.67336	536.24604	23.33686

Базовый метод ближайшего соседа ненадежен, часто не находя цикл, тогда как модификация и отжиг всегда дают решение. На больших графах сосед масштабируется лучше, чем отжиг. Рекомендуется модификация ближайшего соседа для оптимальной точности и скорости; сверхбыстрый отжиг — если скорость важнее качества.

7 Вывод

Алгоритм имитации отжига для задачи коммивояжера прост в реализации, но работает медленно и неточно на больших графах. На малых графах с 4 вершинами он дает короткий путь (112), но это объясняется малым числом возможных маршрутов и начальным маршрутом от поиска в глубину, а не эффективностью алгоритма. На графах с 500 вершинами путь длинный (11740), а время выполнения достигает 536 секунд. Ускоренная версия отжига сокращает время до 23 секунд, но увеличивает длину пути до 12464. Модифицированный метод ближайшего соседа точнее (путь 863), быстрее (6.67 секунды) и не сложнее в реализации. Таким образом, для решения задачи коммивояжера лучше использовать модифицированный метод ближайшего соседа, а имитация отжига подходит только для учебных целей или задач с гибкими настройками.

8 Полезные ссылки

- Репозиторий с лабораторными работами за 4-й семестр

9 Листинг программы

9.1 main.py

```
from colorGUI import LabApp
from graph import Graph
import random
import sys

sys.setrecursionlimit(250000)
RANDOM_SEED = 0

def create_graph(num_nodes, num_edges, canvas_width, canvas_height):
    """
    :param num_nodes:
    :param num_edges:
    :param canvas_width:
    :param canvas_height:
    :return:
    """
    random.seed(RANDOM_SEED)

    graph = Graph(directed=True)

    cols = int(num_nodes**0.5)
    rows = (num_nodes + cols - 1) // cols
    padding = 30

    x_step = (canvas_width - 2*padding) / (cols-1) if cols > 1 else 0
    y_step = (canvas_height - 2*padding) / (rows-1) if rows > 1 else 0

    nodes = []
    for i in range(num_nodes):
        col = i % cols
        row = i // cols
        x = padding + col * x_step + random.randint(-10, 10)
        y = padding + row * y_step + random.randint(-10, 10)
        node = graph.add_node(i)
```

```

        node.x = x
        node.y = y
        nodes.append(node)

all_possible_edges = [(i, j) for i in range(num_nodes)
                       for j in range(num_nodes) if i != j]
selected_edges = random.sample(all_possible_edges, num_edges)

for from_idx, to_idx in selected_edges:
    weight = random.randint(1, 50)
    graph.add_edge(nodes[from_idx], nodes[to_idx], weight)

return graph

def main():
    # graph = create_graph(num_nodes=4, num_edges=8, canvas_width=676,
    #                       ↪ canvas_height=300)
    graph = create_graph(num_nodes=20, num_edges=186, canvas_width=676,
    #                     ↪ canvas_height=300)
    # graph = create_graph(num_nodes=100, num_edges=3395, canvas_width=676,
    #                       ↪ canvas_height=300)
    # graph = create_graph(num_nodes=500, num_edges=100000,
    #                       ↪ canvas_width=676, canvas_height=300)
    # graph = Graph()

    app = LabApp(graph)
    app.attributes('-fullscreen', True)
    app.mainloop()

if __name__ == "__main__":
    main()

```

9.2 DFS.py

```
from typing import Tuple, Optional, List
from graph import Graph, Node, Edge

def dfs_hamiltonian_cycle(graph: Graph, start_node: Optional[Node] = None)
    ↪ -> Tuple[Graph, int, str, str]:
    """
        DFS.

    :param graph:
        .
    :param start_node:
        .
    :return:
        :
        -
        -
        -
        - "A-B-C-A"
    """
    if not graph.nodes:
        return Graph(), 0, " : ", ""

    if start_node is None:
        start_node = graph.nodes[0]
    elif start_node not in graph.nodes:
        return Graph(), 0, " : ", ""

    result_path = []
    result_weight = 0

    def backtrack(current_node: Node, path: List[Node], visited: set,
        ↪ weight: int) -> bool:
        nonlocal result_path, result_weight

        new_path = path + [current_node]
        new_visited = visited | {current_node}

        if len(new_visited) == len(graph.nodes):
            for edge in current_node.outgoing_edges:
                if edge.to_node == start_node:
```

```

        result_path = new_path + [start_node]
        result_weight = weight + edge.weight
        return True
    return False

for edge in current_node.outgoing_edges:
    neighbor = edge.to_node
    if neighbor not in new_visited:
        if backtrack(neighbor, new_path, new_visited, weight +
            ↪ edge.weight):
            return True

return False

found = backtrack(start_node, [], set(), 0)

if not found or not result_path:
    return Graph(), 0, " ", ""

result_graph = Graph(directed=True)
node_map = {}

for node in result_path:
    if node.value not in node_map:
        new_node = result_graph.add_node(node.value)
        new_node.x, new_node.y = node.x, node.y
        node_map[node.value] = new_node

total_weight = 0
path_values = []
for i in range(len(result_path)-1):
    from_node = result_path[i]
    to_node = result_path[i+1]

    edge = next(e for e in from_node.outgoing_edges if e.to_node ==
        ↪ to_node)
    total_weight += edge.weight

    result_graph.add_edge(node_map[from_node.value],
        ↪ node_map[to_node.value], edge.weight)

```

```
    path_values.append(str(from_node.value))

    path_values.append(str(start_node.value))
    path_str = "-".join(path_values)

    return result_graph, total_weight, " ", path_str
```

9.3 simulated_annealing.py

```
from graph import Graph, Node, Edge
from typing import Tuple, Optional, List
import random
import math
from DFS import dfs_hamiltonian_cycle

def calculate_path_length(graph: Graph, path: List[Node]) -> int:
    """
    .

    :param graph:
    :param path:
    :return:
    """
    if len(path) != len(graph.nodes) + 1 or path[0] != path[-1]:
        return float('inf')

    total = 0
    for i in range(len(path) - 1):
        from_node = path[i]
        to_node = path[i + 1]

        #
        original_from_node = next(n for n in graph.nodes if n.value ==
            ↪ from_node.value)
        original_to_node = next(n for n in graph.nodes if n.value ==
            ↪ to_node.value)

        #
        edge = next(
            (e for e in original_from_node.outgoing_edges if e.to_node ==
            ↪ original_to_node),
            None
        )

        if not edge:
            return float('inf')
```

```

        total += edge.weight

    return total

def get_initial_path(graph: Graph) -> List[Node]:
    """
        DFS.

    :param graph:
    :return:
    :raises ValueError:    DFS
    """
    result_graph, _, status, path_str = dfs_hamiltonian_cycle(graph)

    if status != "    ":
        raise ValueError("DFS    ")

    path = []
    for val in path_str.split("-")[:-1]:
        original_node = next(n for n in graph.nodes if str(n.value) == val)
        node_copy = Node(original_node.value)
        node_copy.x = original_node.x
        node_copy.y = original_node.y
        node_copy.outgoing_edges = original_node.outgoing_edges
        path.append(node_copy)

    path.append(path[0])
    return path

def two_opt_swap(path: List[Node], i: int, k: int) -> List[Node]:
    """
        2-opt swap    .

    :param path:
    :param i:
    :param k:
    :return:

```

```

"""
new_path = path.copy()
new_path[i:k+1] = reversed(new_path[i:k+1])

for idx in range(len(new_path)):
    new_path[idx].x = path[idx].x
    new_path[idx].y = path[idx].y

return new_path

def simulated_annealing(
    graph: Graph,
    initial_temp: Optional[float] = None,
    min_temp: float = 1,
    cooling_rate: float = 0.95,
    fast_annealing: bool = False
) -> Tuple[Graph, int, str, str]:
    """
        DFS        -.

    :param graph:
    :param initial_temp:
    :param min_temp:
    :param cooling_rate:
    :param fast_annealing:
    :return:
    """
    try:
        current_path = get_initial_path(graph)
    except ValueError as e:
        return Graph(), 0, f"        : {str(e)}", ""

    current_length = calculate_path_length(graph, current_path)
    if current_length == float('inf'):
        return Graph(), 0, "        ", ""

    best_path = current_path.copy()
    best_length = current_length
    temp = initial_temp if initial_temp is not None else 100.0 #

```

```

num_of_iteration = 0

while temp >= min_temp:
    num_of_iteration += 1
    print(f"      : {num_of_iteration},      : {temp},      :
          ↪ {current_length}")

    a, b = sorted(random.sample(range(1, len(current_path)-1), 2))
    new_path = two_opt_swap(current_path, a, b)
    new_length = calculate_path_length(graph, new_path)

    if new_length == float('inf'):
        continue

    delta = new_length - current_length
    accept_prob = math.exp(-delta / (temp * (1 + 0.1 * current_length /
          ↪ best_length)))

    if delta < 0 or random.random() < accept_prob:
        current_path = new_path
        current_length = new_length

        if current_length < best_length:
            best_path = current_path.copy()
            best_length = current_length

    if fast_annealing:
        temp = initial_temp / (1 + num_of_iteration) if initial_temp is
          ↪ not None else 100.0 / (1 + num_of_iteration)
    else:
        temp *= cooling_rate

result_graph = Graph(directed=True)
node_map = {}

for node in graph.nodes:
    new_node = result_graph.add_node(node.value)
    new_node.x = node.x
    new_node.y = node.y
    node_map[node.value] = new_node

```

```

total_weight = 0
path_values = []
for i in range(len(best_path)-1):
    from_val = best_path[i].value
    to_val = best_path[i+1].value

    original_from = next(n for n in graph.nodes if n.value == from_val)
    original_to = next(n for n in graph.nodes if n.value == to_val)

    edge = next((e for e in original_from.outgoing_edges if e.to_node
        ↪ == original_to), None)
    if not edge:
        return Graph(), 0, " ", ""

    total_weight += edge.weight
    result_graph.add_edge(node_map[from_val], node_map[to_val],
        ↪ edge.weight)
    path_values.append(str(from_val))

path_str = "-".join(path_values + [str(best_path[0].value)])

return result_graph, total_weight, " ", path_str

```

9.4 colorGUI.py

```
import tkinter as tk
from tkinter import simpledialog, messagebox, ttk
import math
import time
import platform
from graph import Graph, Edge
from simulated_annealing import simulated_annealing

class LabApp(tk.Tk):
    def __init__(self, graph):
        super().__init__()
        self.title(" ")
        self.geometry("1250x700")

        self.columnconfigure(0, weight=1, minsize=200)
        self.columnconfigure(1, weight=2, minsize=700)
        self.columnconfigure(2, weight=1, minsize=350)
        self.rowconfigure(0, weight=1)

        self.graph = graph
        self.node_clicked = None
        self.answer_graph = None
        self.tree = None
        self.path_length_entry = None
        self.path_text = None
        self.output_canvas = None
        self.time_entry = None #

        self._setup_styles()
        self.create_widgets()
        self.draw_initial_graph()

    def _setup_styles(self):
        """ """
        self.BG_COLOR = "#F0F0F0"
        self.ACCENT_1 = "#FF6B6B"
        self.ACCENT_2 = "#4ECDC4"
```

```

self.DARK_TEXT = "#2D3436"
self.LIGHT_TEXT = "#FFFFFF"
self.PANEL_COLOR = "#FFEAA7"
self.BTN_COLOR = "#55EFC4"
self.CANVAS_BG = "#DFE6E9"

style = ttk.Style()
style.theme_use("default")
style.configure("Treeview.Heading",
                background=self.ACCEENT_1,
                foreground=self.LIGHT_TEXT,
                font=('Helvetica', 10, 'bold'))
style.configure("Treeview",
                background="#FFFFFF",
                fieldbackground="#FFFFFF",
                foreground=self.DARK_TEXT)
style.map("Treeview", background=[('selected', self.ACCEENT_2)])

def create_widgets(self):
    """ """
    left_frame = tk.LabelFrame(
        self,
        padx=10,
        pady=10,
        text="      ",
        bg=self.PANEL_COLOR,
        fg=self.DARK_TEXT,
        font=('Helvetica', 12, 'bold')
    )
    left_frame.grid(row=0, column=0, sticky="nsew")

    #
    param_frame = tk.Frame(left_frame, bg=self.PANEL_COLOR)
    param_frame.pack(fill="x", pady=5)

    params = [
        ("      :", "initial_temp", "10000"),
        ("      :", "min_temp", "1"),
        ("      :", "cooling_rate", "0.95"),
    ]

```

```

for i, (label_text, var_name, default) in enumerate(params):
    tk.Label(
        param_frame,
        text=label_text,
        bg=self.PANEL_COLOR,
        fg=self.DARK_TEXT,
        font=('Helvetica', 10)
    ).grid(row=i, column=0, sticky="w", padx=5, pady=2)

    entry = tk.Entry(param_frame, width=15)
    entry.insert(0, default)
    entry.grid(row=i, column=1, padx=5, pady=2)
    setattr(self, f"{var_name}_entry", entry)

self.fast_annealing_var = tk.BooleanVar()
tk.Checkbutton(
    param_frame,
    text="          ",
    variable=self.fast_annealing_var,
    bg=self.PANEL_COLOR,
    font=('Helvetica', 10)
).grid(row=len(params), columnspan=2, pady=5)

#
tk.Button(
    left_frame,
    text="          ",
    command=self.start_algorithm,
    bg=self.BTN_COLOR,
    fg=self.DARK_TEXT,
    activebackground=self.ACCENT_2,
    font=('Helvetica', 12, 'bold')
).pack(fill="x", pady=10)

#          ()
tk.Label(
    left_frame,
    text="          ():",
    bg=self.PANEL_COLOR,

```

```

        fg=self.DARK_TEXT,
        font=('Helvetica', 10)
    ).pack(anchor="w")

self.time_entry = tk.Entry(
    left_frame,
    state="readonly",
    bg="#FFFFFF",
    fg=self.ACCEENT_1,
    font=('Helvetica', 12, 'bold'),
    justify="center",
    readonlybackground="#FFFFFF"
)
self.time_entry.pack(fill="x", pady=2)
self.time_entry.bind("<Button-3>", self.show_context_menu)
self.time_entry.bind("<Control-c>", lambda e: self.copy_time())
if platform.system() == "Darwin":
    self.time_entry.bind("<Command-c>", lambda e: self.copy_time())

#
tk.Label(
    left_frame,
    text="          :",
    bg=self.PANEL_COLOR,
    fg=self.DARK_TEXT,
    font=('Helvetica', 10)
).pack(anchor="w")

self.path_text = tk.Text(
    left_frame,
    height=5,
    state="disabled",
    bg="#FFFFFF",
    fg="#2D3436",
    font=('Courier New', 14)
)
self.path_text.pack(fill="both", pady=5, expand=True)

# self.path_text = tk.Text(
#     left_frame,

```

```

#     height=5,
#     state="disabled",
#     bg="#FFFFFF",
#     fg=self.DARK_TEXT,
#     font=('Courier New', 14),
#     wrap=tk.NONE,
#     selectbackground=self.ACCENT_2,
#     inactiveselectbackground=self.ACCENT_2,
#     exportselection=1,
#     highlightthickness=0,
#     borderwidth=2,
#     relief="solid"
# )
# self.path_text.pack(fill="both", pady=5, expand=True)
# self.path_text.bind("<Button-3>", self.show_text_context_menu)
# self.path_text.bind("<Control-c>", lambda e:
#     ↪ self.copy_path_text())
# self.path_text.bind("<Control-Insert>", lambda e:
#     ↪ self.copy_path_text())
# if platform.system() == "Darwin":
#     self.path_text.bind("<Command-c>", lambda e:
#         ↪ self.copy_path_text())

tk.Label(
    left_frame,
    text="          :",
    bg=self.PANEL_COLOR,
    fg=self.DARK_TEXT,
    font=('Helvetica', 10)
).pack(anchor="w")

self.path_length_entry = tk.Entry(
    left_frame,
    state="readonly",
    bg="#FFFFFF",
    fg=self.ACCENT_1,
    font=('Helvetica', 12, 'bold'),
    justify="center",
    readonlybackground="#FFFFFF"
)

```

```

self.path_length_entry.pack(fill="x", pady=2)
self.path_length_entry.bind("<Button-3>", self.show_context_menu)
self.path_length_entry.bind("<Control-c>", lambda e:
    ↪ self.copy_path_length())
if platform.system() == "Darwin":
    self.path_length_entry.bind("<Command-c>", lambda e:
        ↪ self.copy_path_length())

#
center_frame = tk.Frame(self, bg=self.BG_COLOR)
center_frame.grid(row=0, column=1, sticky="nsew")

#
input_graph_frame = tk.LabelFrame(
    center_frame,
    text="          ",
    padx=10,
    pady=10,
    bg=self.BG_COLOR,
    fg=self.DARK_TEXT,
    font=('Helvetica', 12, 'bold')
)
input_graph_frame.pack(fill="both", expand=True, side=tk.TOP)

self.input_canvas = tk.Canvas(
    input_graph_frame,
    bg=self.CANVAS_BG,
    highlightthickness=0
)
self.input_canvas.pack(fill="both", expand=True)
self.input_canvas.bind("<Button-1>", self.on_canvas_click)

#
tk.Button(
    center_frame,
    text="          ",
    command=self.clear_graph,
    bg=self.ACCENT_1,
    fg=self.DARK_TEXT,
    activebackground="#FF8787",

```

```

        font=('Helvetica', 10),
        height=2
    ).pack(fill="x", pady=5)

#
output_graph_frame = tk.LabelFrame(
    center_frame,
    text="      ",
    padx=10,
    pady=10,
    bg=self.BG_COLOR,
    fg=self.DARK_TEXT,
    font=('Helvetica', 12, 'bold')
)
output_graph_frame.pack(fill="both", expand=True, side=tk.TOP)

self.output_canvas = tk.Canvas(
    output_graph_frame,
    bg=self.CANVAS_BG,
    highlightthickness=0
)
self.output_canvas.pack(fill="both", expand=True)

#
right_frame = tk.LabelFrame(
    self,
    text="      ",
    padx=10,
    pady=10,
    bg=self.PANEL_COLOR,
    fg=self.DARK_TEXT,
    font=('Helvetica', 12, 'bold')
)
right_frame.grid(row=0, column=2, sticky="nsew")

self.tree = ttk.Treeview(
    right_frame,
    columns=("from", "to", "weight", "used"),
    show="headings",
    height=25

```

```

)

for col in ("from", "to", "weight", "used"):
    self.tree.heading(col, text=col.capitalize())
    self.tree.column(col, width=80, anchor="center")

scrollbar = ttk.Scrollbar(
    right_frame,
    orient="vertical",
    command=self.tree.yview
)
self.tree.configure(yscrollcommand=scrollbar.set)

self.tree.pack(side="left", fill="both", expand=True)
scrollbar.pack(side="right", fill="y")

#
def copy_to_clipboard(self, text):
    self.clipboard_clear()
    self.clipboard_append(text)
    if platform.system() == "Darwin":
        self.update()

def show_context_menu(self, event):
    menu = tk.Menu(self, tearoff=0)
    menu.add_command(label="      ", command=self.copy_time)
    menu.tk.call("tk_popup", menu, event.x_root, event.y_root)

def show_text_context_menu(self, event):
    menu = tk.Menu(self, tearoff=0)
    menu.add_command(label="      ", command=self.copy_path_text)
    menu.tk.call("tk_popup", menu, event.x_root, event.y_root)

def copy_time(self):
    text = self.time_entry.get()
    self.copy_to_clipboard(text)

def copy_path_text(self):
    try:
        text = self.path_text.get("sel.first", "sel.last")

```

```

except tk.TclError:
    text = self.path_text.get("1.0", "end")
    self.copy_to_clipboard(text)

def copy_path_length(self):
    text = self.path_length_entry.get()
    self.copy_to_clipboard(text)

def on_canvas_click(self, event):
    """ """
    x, y = event.x, event.y
    node = self._find_node_at(x, y)

    if node:
        if self.node_clicked:
            self._create_edge(self.node_clicked, node)
            self.node_clicked = None
        else:
            self.node_clicked = node
    else:
        if not self.node_clicked:
            self._create_node(x, y)

def _find_node_at(self, x, y):
    """ """
    for node in self.graph.nodes:
        if abs(node.x - x) < 15 and abs(node.y - y) < 15:
            return node
    return None

def _create_node(self, x, y):
    """ """
    node = self.graph.add_node(len(self.graph.nodes))
    node.x = x
    node.y = y
    self._draw_node(node, self.input_canvas, "red")
    self.update_edges_table()

def _create_edge(self, from_node, to_node):
    """ """

```

```

if from_node == to_node:
    return

weight = simpldialog.askinteger(
    "      ",
    f"      {from_node.value} → {to_node.value}:"
)

if weight:
    self.graph.add_edge(from_node, to_node, weight)
    self._draw_edge(Edge(from_node, to_node, weight),
        → self.input_canvas)
    self.update_edges_table()

def _draw_node(self, node, canvas, color):
    """      """
    x, y = node.x, node.y
    canvas.create_oval(x-15, y-15, x+15, y+15, fill=color, outline="")
    canvas.create_text(x, y, text=str(node.value), fill="white")

def _draw_edge(self, edge, canvas):
    """      """
    from_node = edge.from_node
    to_node = edge.to_node

    dx = to_node.x - from_node.x
    dy = to_node.y - from_node.y
    angle = math.atan2(dy, dx)

    offset = 15
    x1 = from_node.x + offset * math.cos(angle)
    y1 = from_node.y + offset * math.sin(angle)
    x2 = to_node.x - offset * math.cos(angle)
    y2 = to_node.y - offset * math.sin(angle)

    canvas.create_line(x1, y1, x2, y2, arrow=tk.LAST, width=2)
    mid_x = (x1 + x2) / 2
    mid_y = (y1 + y2) / 2
    canvas.create_text(mid_x, mid_y, text=str(edge.weight))

```

```

def draw_initial_graph(self):
    """ """
    for node in self.graph.nodes:
        self._draw_node(node, self.input_canvas, "red")

    for edge in self.graph.edges:
        self._draw_edge(edge, self.input_canvas)

def clear_graph(self):
    """ """
    self.graph = Graph(directed=True)
    self.input_canvas.delete("all")
    self.output_canvas.delete("all")
    self.update_edges_table()

def start_algorithm(self):
    """ """
    try:
        params = {
            'initial_temp': float(self.initial_temp_entry.get()),
            'min_temp': float(self.min_temp_entry.get()),
            'cooling_rate': float(self.cooling_rate_entry.get()),
            'fast_annealing': self.fast_annealing_var.get()
        }

        if params['cooling_rate'] >= 1 or params['cooling_rate'] <= 0:
            raise ValueError("0 1")

    except ValueError as e:
        messagebox.showerror(" ", f" : {str(e)}")
        return

    try:
        print("start")
        start_time = time.time() #
        result = simulated_annealing(
            self.graph,
            initial_temp=params['initial_temp'],
            min_temp=params['min_temp'],
            cooling_rate=params['cooling_rate'],

```

```

        fast_annealing=params['fast_annealing']
    )
    end_time = time.time() #
    execution_time = end_time - start_time

    self.time_entry.configure(state="normal")
    self.time_entry.delete(0, tk.END)
    self.time_entry.insert(0, f"{execution_time:.4f}")
    self.time_entry.configure(state="readonly")

    self.answer_graph, length, status, path = result
    self._update_results(length, status, path)
    self._draw_result_graph()
    self.update_edges_table()
    print(path)

except Exception as e:
    messagebox.showerror(" ", str(e))

def _update_results(self, length, status, path):
    """ """
    self.path_text.configure(state="normal")
    self.path_text.delete("1.0", tk.END)
    self.path_text.insert(tk.END, path)
    self.path_text.configure(state="disabled")

    self.path_length_entry.configure(state="normal")
    self.path_length_entry.delete(0, tk.END)
    self.path_length_entry.insert(0, str(length))
    self.path_length_entry.configure(state="readonly")

    messagebox.showinfo(" ", status)

def _draw_result_graph(self):
    """ """
    self.output_canvas.delete("all")

    for node in self.answer_graph.nodes:
        self._draw_node(node, self.output_canvas, "blue")

```

```

        for edge in self.answer_graph.edges:
            self._draw_edge(edge, self.output_canvas)

def update_edges_table(self):
    """ """

    for item in self.tree.get_children():
        self.tree.delete(item)

    used_edges = set()
    if self.answer_graph:
        for edge in self.answer_graph.edges:
            key = (edge.from_node.value, edge.to_node.value, edge.weight)
            used_edges.add(key)

    sorted_edges = sorted(
        self.graph.edges,
        key=lambda edge: (edge.from_node.value, edge.to_node.value)
    )

    for edge in sorted_edges:
        from_val = edge.from_node.value
        to_val = edge.to_node.value
        weight = edge.weight
        is_used = (from_val, to_val, weight) in used_edges

        self.tree.insert("", "end", values=(
            from_val,
            to_val,
            weight,
            " " if is_used else " "
        ))

if __name__ == "__main__":
    initial_graph = Graph(directed=True)
    app = LabApp(initial_graph)
    app.mainloop()

```
